Laboratório Funcional · Aula 12

Vinicius Girato Moresqui · RA 26014779-2 · Turma ESOFT6S-C
Gerado em 16/10/2026, 20:44:02 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1 - def agora(): return datetime.now()
R: Ela não é pura porque depende do relógio do computador. Como ela não recebe nenhum parâmetro e olha a hora atual na hora em que é executada, o resultado muda o tempo todo.

2 - def poe(lista, x): lista.append(x); return lista
R: Ela é impura porque causa um efeito colateral direto. Em vez de criar uma lista nova com o elemento extra, ela mexe e altera a lista original que foi passada como argumento lá de fora, modificando o estado dela durante o processo.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: 1: A-1 A-2 (pois a função impura incrementa o contador global a cada chamada).
Linha 4: 4: 14 (calculado como f(g(4)), onde g(4) = 12 e f(12) = 14).
Linha 7: 7: [7, 3, 9] [3, 7, 9] (mostra que sorted() cria uma nova lista sem alterar a original).

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
```

```
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```



1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque a função depende de uma variável global (contador) que fica fora dela e muda de estado a cada execução. Isso dificulta os testes porque o resultado de uma chamada deixa de depender apenas do argumento fornecido, exigindo que você conheça todo o histórico de execuções anteriores do programa para saber o que a função vai retornar (violando a transparência referencial).

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Cálculos à mão:

• compor(f, g)(4) = f(g(4))
1. g(4) = 3 * 4 = 12
2. f(12) = 12 + 2 = 14
Resultado: 14

• compor(g, f)(4) = g(f(4))
1. f(4) = 4 + 2 = 6
2. g(6) = 3 * 6 = 18
Resultado: 18

Explicação:
Os valores deram diferentes porque a composição de funções não é comutativa ($f \circ g \neq g \circ f$). A ordem em que as funções são encadeadas altera qual operação é realizada primeiro, mudando o valor que serve de entrada para a segunda função.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) gera e retorna uma nova lista ordenada, deixando a lista original inalterada.

notas.sort() modifica diretamente a lista original na memória (in-place) e retorna None.

O que segue o estilo funcional é o sorted(notas), pois ele respeita o princípio da imutabilidadade e não causa efeitos colaterais no estado externo dos dados.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

retorna o primeiro elemento da lista

Previsão: (cdr '(A B C))

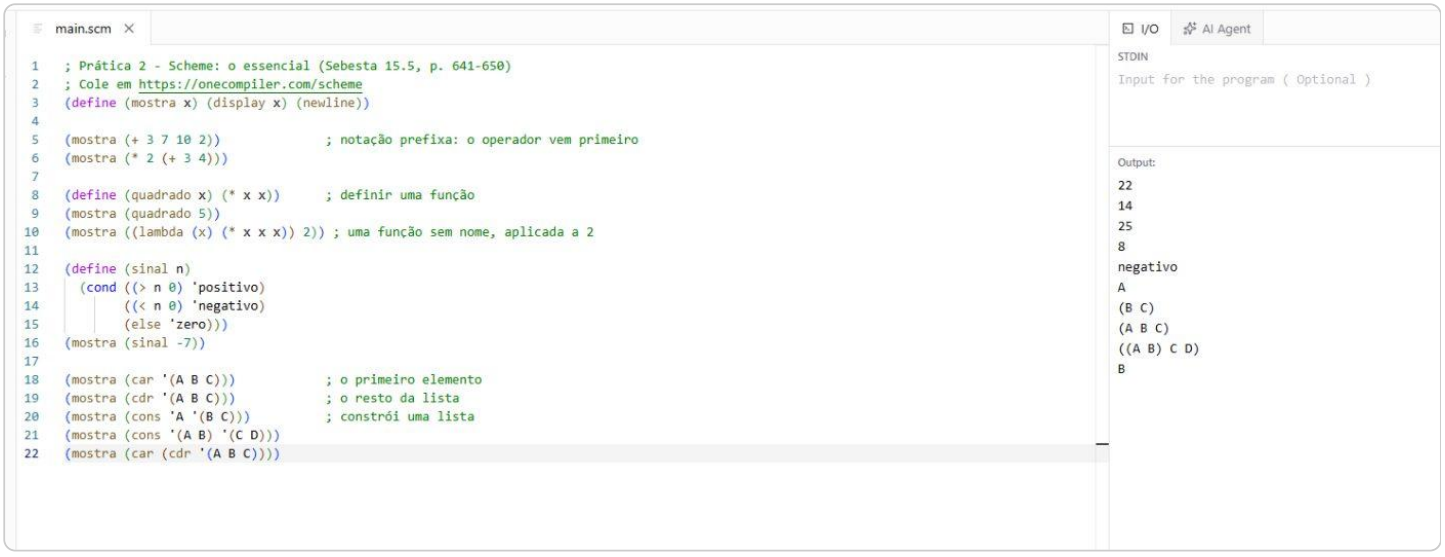
retorna o restante da lista após o primeiro elemento

Previsão: (cons '(A B) '(C D))

((A B) C D) (insere a lista (A B) no início da lista (C D))

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```



2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo ('), que significa QUOTE, serve para indicar ao Scheme que a lista deve ser tratada como dados literais e não avaliada como uma chamada de função. Quando removemos o apóstrofo e escrevemos (car (A B C)), o interpretador tenta executar A como se fosse uma função chamando os argumentos B e C, gerando um erro de execução (pois A não é uma função definida)

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

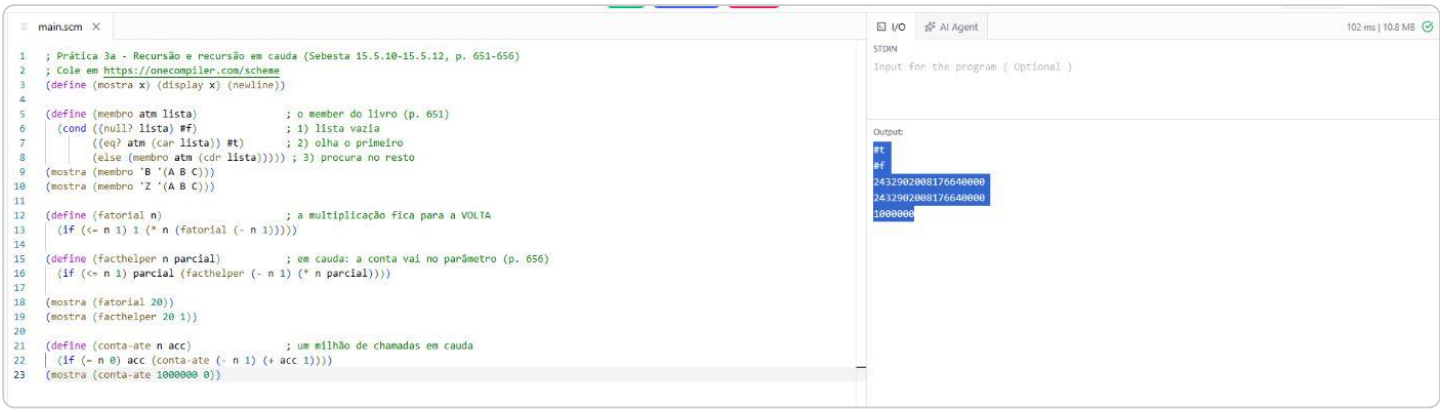
não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme vai funcionar perfeitamente, pois a linguagem otimiza chamadas em cauda (Tail Call Optimization). Já no Python, a execução deve falhar gerando um estouro de pilha (RecursionError), visto que o interpretador do Python não faz essa otimização por padrão e impõe um limite máximo de recursões.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

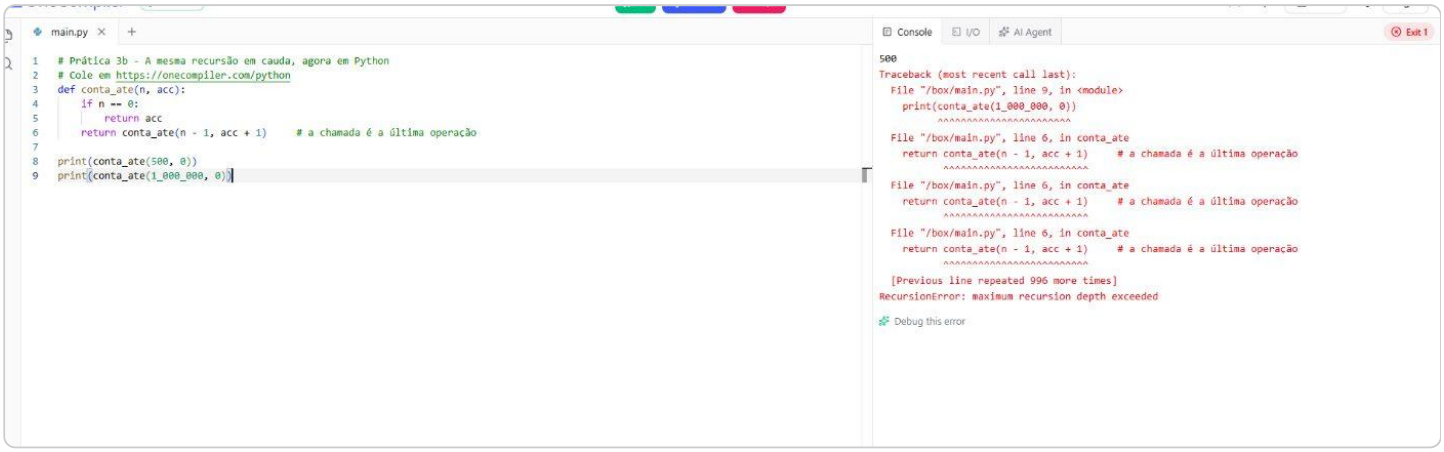


Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
print(conta_ate(1_000_000, 0))
Exit 1
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
```

```
          ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded

Debug thi
```



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, o número de operações pendentes (e o tamanho das expressões na pilha) cresce linearmente a cada chamada, porque as multiplicações ficam guardadas esperando a função retornar para serem resolvidas. Na recursão em cauda, como o resultado parcial é levado adiante por um parâmetro acumulador e a chamada recursiva é a última operação, nada fica esperando na pilha, mantendo o consumo de espaço constante.

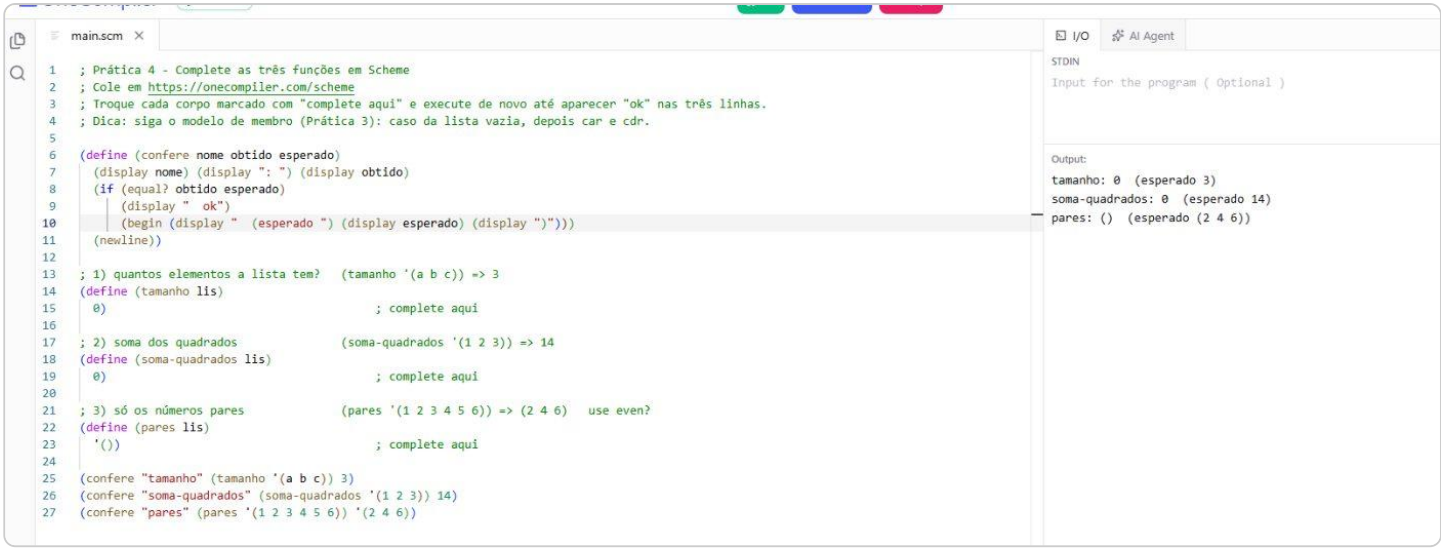
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme realizou um milhão de chamadas porque implementa a otimização de chamada em cauda (Tail Call Optimization), reaproveitando o mesmo registro de ativação da pilha em vez de criar um novo para cada iteração. O Python, por sua vez, não realiza essa otimização e mantém o limite padrão de segurança da pilha de chamadas (geralmente em torno de 1.000 frames), fazendo com que um milhão de chamadas exceda esse limite e lance um RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))
```



4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
; 1) quantos elementos a lista tem?   (tamanho '(a b c)) ⇒ 3
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

; 2) soma dos quadrados               (soma-quadrados '(1 2 3)) ⇒ 14
(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis))))))

; 3) só os números pares              (pares '(1 2 3 4 5 6)) ⇒ (2 4 6)   use even?
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

Parte mais difícil: Lembrar de usar a função cons corretamente para reconstruir a lista de números pares sem perder a estrutura e o encadeamento das chamadas recursivas.

Uso de null?, car e cdr:

Em todas as funções, usamos null? no caso base para verificar se a lista chegou ao fim (retornando 0 ou ''()).

O car foi usado para acessar e operar sobre o primeiro elemento atual da lista (como multiplicar por ele mesmo em soma-quadrados ou testar a paridade em pares).

O cdr foi utilizado em todas as chamadas recursivas para avançar a lista, descartando o primeiro elemento já processado.

Prática 5: Haskell: padrões, compreensões e preguiça

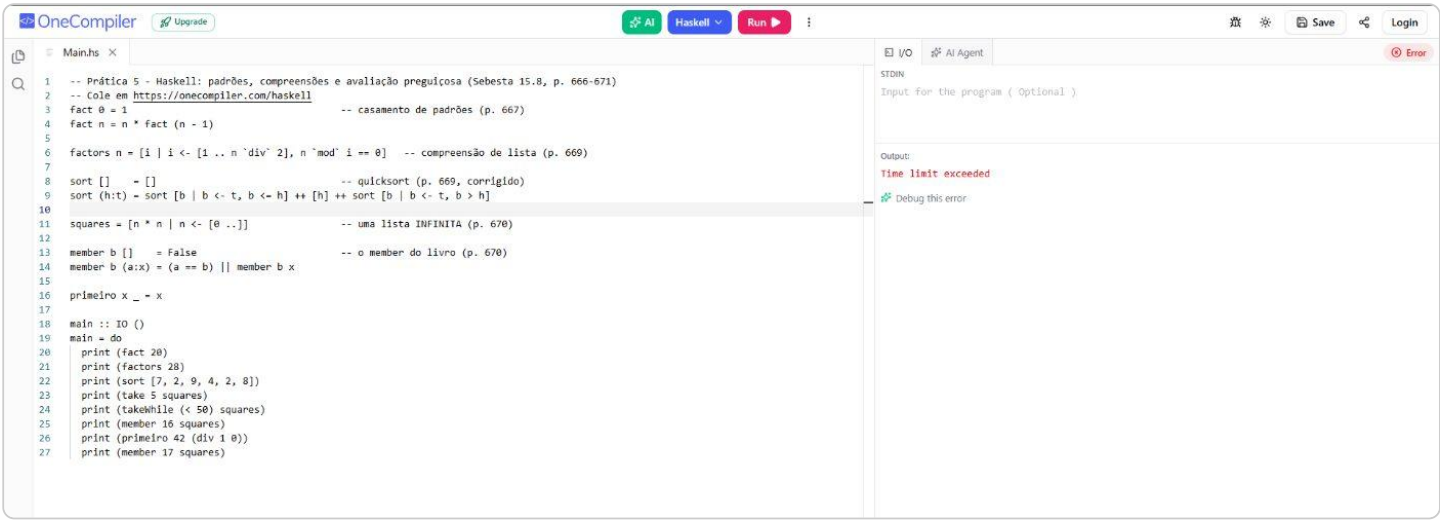
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

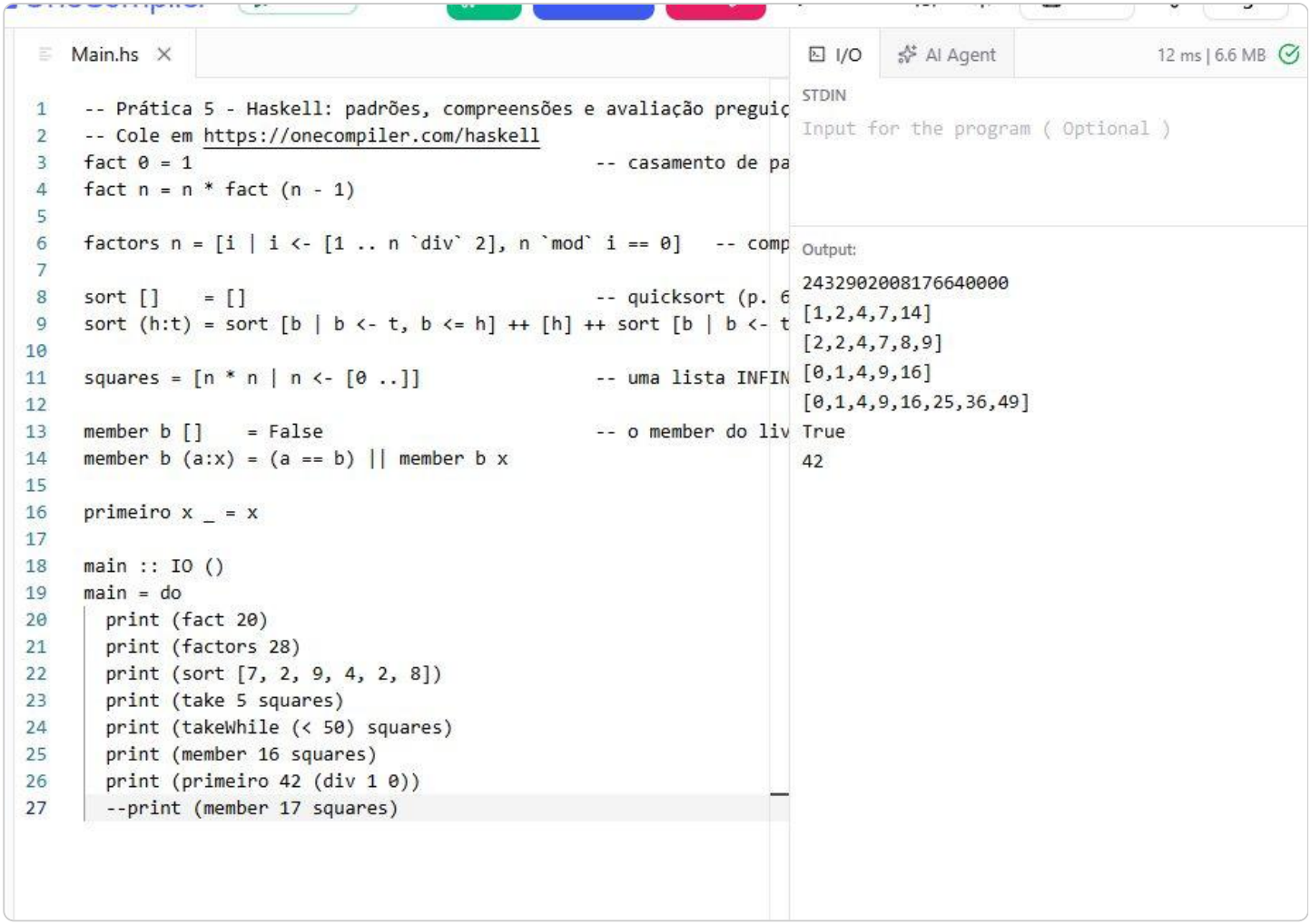
O take 5 squares deve imprimir os 5 primeiros quadrados da lista infinita (ou seja, [0, 1, 4, 9, 16]). Já primeiro 42 (div 1 0) deve retornar 42 com sucesso, sem causar erro de divisão por zero, devido à avaliação preguiçosa do Haskell, que ignora o parâmetro não utilizado.

Saídas (original e com member 17)

```
Sem a linha 17:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Com Linha 17:
Time limit exceeded
```





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue lidar com listas infinitas por causa da avaliação preguiçosa (lazy evaluation): ele só calcula os elementos da lista sob demanda, na medida exata em que eles são solicitados (como os 5 primeiros pedidos pelo take). A função primeiro 42 (div 1 0) não gerou erro de divisão por zero porque a definição da função descarta o segundo argumento (_), fazendo com que o Haskell nunca chegue a avaliar a expressão div 1 0.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O que aconteceu: O programa entra em um loop infinito e trava. Como o 17 não é um quadrado perfeito, a função continua percorrendo a lista infinita de quadrados indefinidamente em busca dele. O teste com 16 terminou porque o 16 faz parte da lista (4 * 4), permitindo que a busca encontrasse o valor e encerrasse.

Como corrigir: Como a lista squares está em ordem estritamente crescente, podemos parar a busca assim que o elemento atual da lista se tornar maior do que o valor que estamos procurando, sabendo que ele não aparecerá mais adiante. O livro (p. 670) sugere parar quando o termo gerado ultrapassar o alvo.

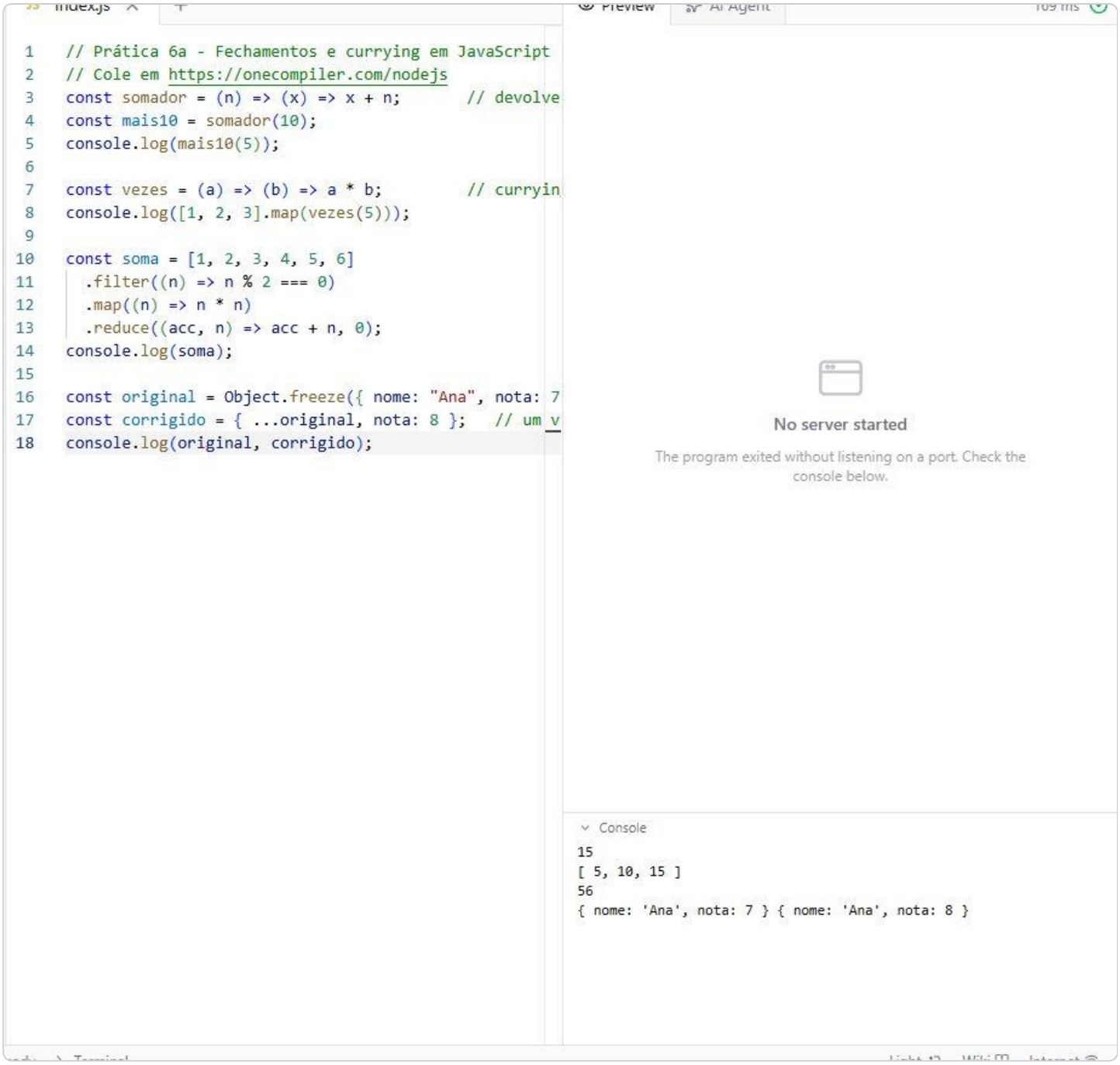
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) vai imprimir 15.
[1, 2, 3].map(vezes(5)) vai imprimir [5, 10, 15].

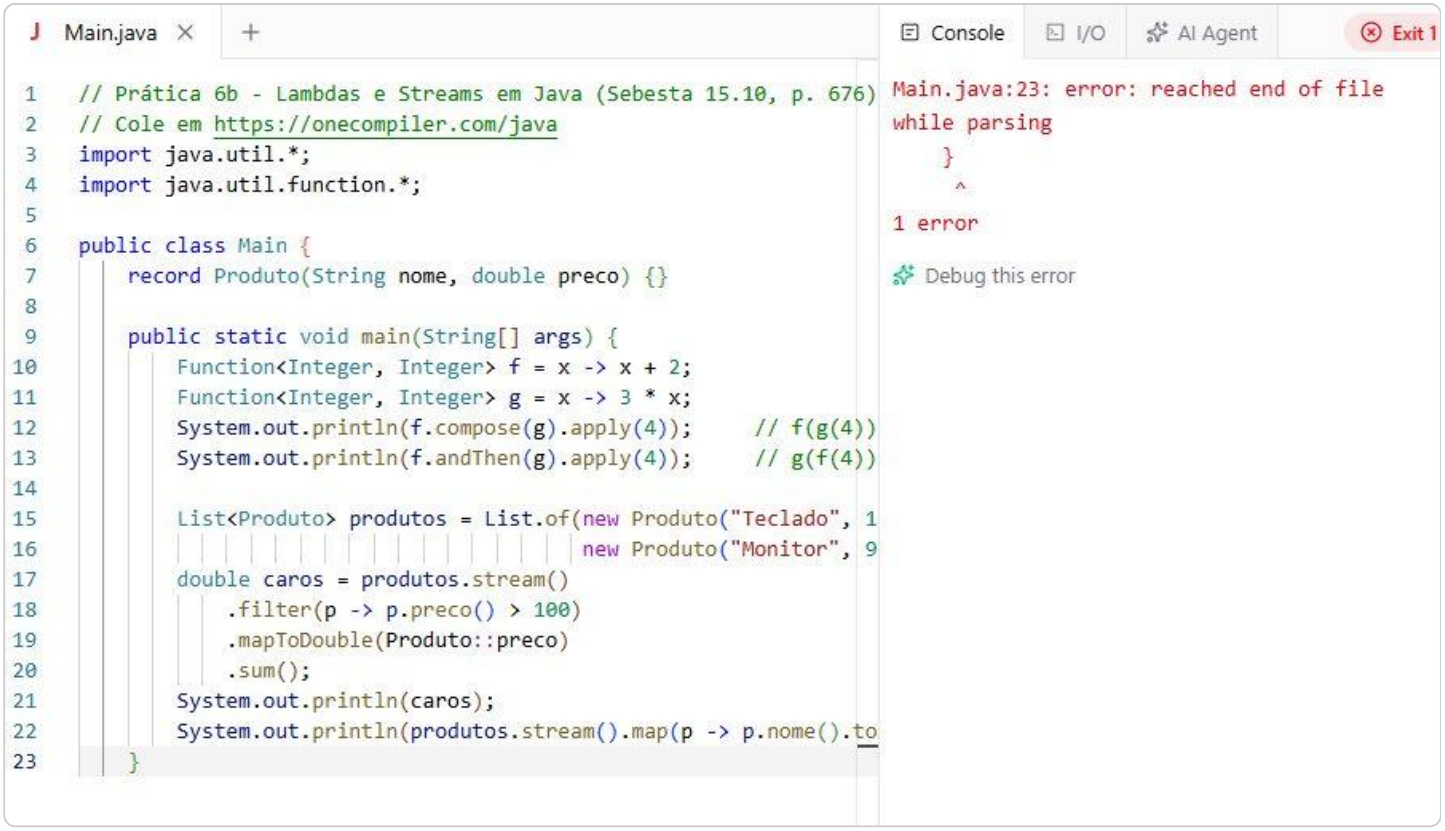
Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```



Saída do Java

```
Main.java:23: error: reached end of file while parsing
    }
     ^
1 error
```



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida lembra do valor 10 devido ao conceito de closure (fechamento). Quando uma função interna é criada no JavaScript, ela mantém uma referência ao seu escopo léxico original (o ambiente onde foi criada). Mesmo após a função externa somador finalizar a sua execução, as variáveis desse escopo continuam guardadas na memória para a função interna utilizar.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

O f.compose(g) aplica primeiro o g e depois o f, calculando f(g(4)) = 14. Já o f.andThen(g) aplica primeiro o f e depois o g, calculando g(f(4)) = 18. Portanto, o que corresponde ao compor(f, g) da Prática 1 é o f.compose(g).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
int soma = nums.stream()
    .filter(n → n % 2 == 0)
    .mapToInt(n → n * n)
    .sum();
System.out.println(soma);
```

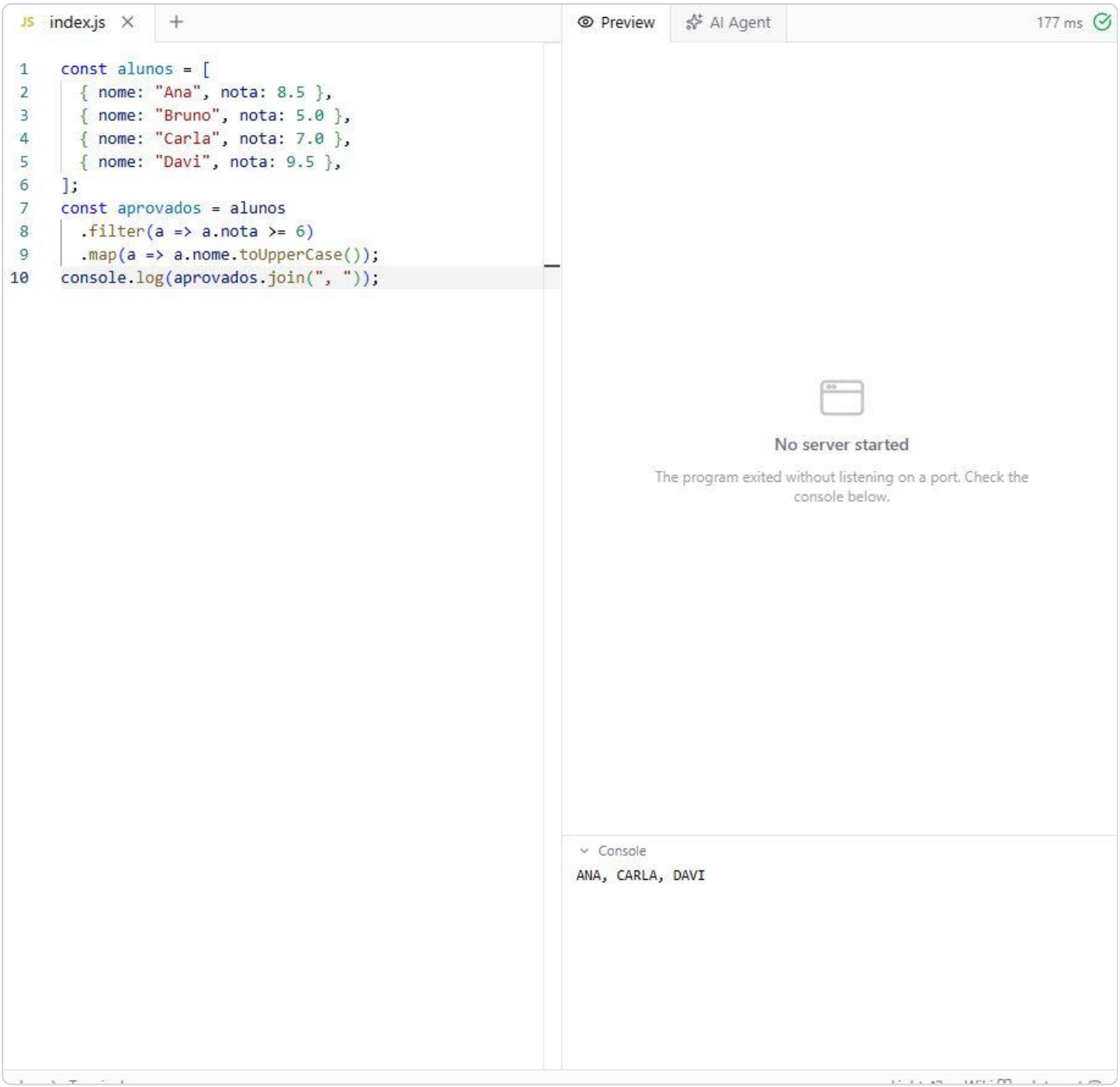
7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
```

```
];
const aprovados = alunos
  .filter(a ⇒ a.nota ⩾ 6)
  .map(a ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Saida: ANA, CARLA, DAVI.
```



7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

precos = [50.0, 120.0, 80.0, 200.0]

com_desconto = [p * 0.9 if p > 100 else p for p in precos]

print(com_desconto)

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender para quem conhece o paradigma funcional, pois lê-se como uma especificação matemática direta ("gere a lista, eleva ao cubo e soma"). Já a versão em C é mais direta para quem vem de linguagens imperativas, focando no controle passo a passo do laço. Aceitaria o custo de desempenho em sistemas de processamento de dados em lotes, aplicações web ou relatórios onde a clareza, a segurança contra efeitos colaterais e a facilidade de manutenção importam mais. Não aceitaria o custo em sistemas de tempo real crítico (softwares embarcados, medicina, automotivo ou games de alta performance) onde o uso estrito de CPU e o tempo de resposta determinístico são essenciais.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais surpreendeu foi ver como a avaliação preguiçosa do Haskell permite lidar com estruturas infinitas sem travar o computador.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está na primeira compreensão de lista da função sort, onde está escrito `b <- h` em vez de `b <= h`. Como `h` é um único elemento (número) e não uma lista, o compilador tenta tratar um número como se fosse uma lista enumerável, gerando um erro de tipo (Type Error) indicando que esperava uma lista e encontrou um tipo numérico. Para corrigir, basta trocar `b <- h` por `b <= h`.